

Final Year Project Progress Diary (BCS-753)

Project Title: AURIX: AI-Powered Automated Security Vulnerability Scanning and Intelligent Context-Aware Auto-Patching VS Code IDE Extension

Project ID: PSIT-AI-2026-08

Team Members: Divyansh, Bhumika, Bhavya, Divyanshi

WEEK 1

Work Done in Current Week:

- Had a team meeting to finalize the architecture and set up our base project files for the frontend, backend, and VS Code extension.
- Created the initial database tables in Supabase and set up a basic login page UI for the web dashboard.

Work to be Done in Next Week:

- Hook up real authentication and create some dummy API routes so we can start testing the connection between the frontend and backend.
-

WEEK 2

Work Done in Current Week:

- Got the login system working! We integrated JWT authentication so users can log in on the web and securely store their session in VS Code.
- On the AI side, we started writing the code parser that reads and understands project files, rather than just grabbing raw text.

Work to be Done in Next Week:

- Build the logic to package local files from VS Code and send them over, and get local vector databases ready for the AI.
-

WEEK 3

Work Done in Current Week:

- Finished the VS Code file packager and added logic to ignore heavy folders like `node_modules` so uploads stay light.

- Set up the main backend server and added mock endpoints to test file uploads.
- Got a local threat intelligence database running to help the AI cross-reference vulnerabilities.

Work to be Done in Next Week:

- Implement frontend polling to check scan statuses, block sensitive files from uploading, and start building the AI's core filtering logic.
-

WEEK 4

Work Done in Current Week:

- Added a progress bar on the web dashboard that automatically polls the backend for scan updates.
- Wrote a local "Secret Guard" in the extension to stop users from accidentally uploading API keys.
- Built the AI's first logic agent to filter out low-confidence bugs so we save compute time.

Work to be Done in Next Week:

- Move away from mock APIs to the real database and start building the isolated Docker environment.
-

WEEK 5

Work Done in Current Week:

- Connected the real backend queues so scanned code is now actually processed by the workers.
- Updated the web dashboard to show scan results in a nice Kanban-style board.
- Successfully got our AI to run its exploit tests inside a secure, temporary Docker container.

Work to be Done in Next Week:

- Handle large file uploads safely and connect our local IDE Secret Guard to the live AI endpoints.
-

WEEK 6

Work Done in Current Week:

- Fixed memory issues by streaming large workspace zip files directly to cloud storage.
- Added syntax highlighting to the web dashboard so the AI-generated code looks readable.
- Got the Red and Blue AI agents working together to automatically write exploits and suggest patches.

Work to be Done in Next Week:

- Optimize our database queries to be faster and teach the AI how to self-correct if its patch fails.
-

WEEK 7

Work Done in Current Week:

- Sped up the backend by rewriting our endpoints for real-time data fetching.
- Added a button on the dashboard so users can manually mark a vulnerability as resolved.
- Built a "Reflexion" loop for the AI—if its code fails in the Docker sandbox, it reads the error and tries again.

Work to be Done in Next Week:

- Set up high-dimensional vector databases and prepare all our documentation for the first Progress Evaluation.
-

WEEK 8

Work Done in Current Week:

- Successfully presented our work for Progress Evaluation 1!
- Wrote scripts to feed OWASP security guidelines into our database for the AI to learn from.
- Updated the VS Code extension to show live execution logs from the backend while a scan is running.

Work to be Done in Next Week:

- Finish converting the security docs into embeddings and start working on the IDE error highlighting.
-

WEEK 9

Work Done in Current Week:

- Finished ingesting the security docs into vector embeddings so the AI can search them quickly.
- Built a slide-out AI chat panel on the web app so users can ask questions about specific bugs.
- Got VS Code to draw red squiggly lines under vulnerable code based on the AI's findings.

Work to be Done in Next Week:

- Add a similarity search feature and build native "Quick Fix" buttons inside VS Code.
-

WEEK 10

Work Done in Current Week:

- Integrated similarity models to make the AI's security advice much more accurate and grounded.
- Fixed some state bugs in the web chat so conversation history isn't lost when navigating pages.
- Added a VS Code "Quick Fix" lightbulb that applies the AI's security patch directly to the file.

Work to be Done in Next Week:

- Speed up the threat search and build an automated GitHub pull request feature.
-

WEEK 11

Work Done in Current Week:

- Deployed a high-speed similarity search on the backend that returns threat context in under 500ms.
- Added a "One-Click PR" button on the web app to automatically fix bugs via GitHub.
- Added "Ghost-Text" in VS Code, allowing users to apply patch suggestions just by hitting Tab.
- Upgraded the AI orchestrator to scan multiple vulnerabilities at the same time.

Work to be Done in Next Week:

- Build webhook receivers for the final AI payloads and design a visualizer for the exploit paths.

WEEK 12

Work Done in Current Week:

- Set up the backend webhooks to catch the final reports once the AI finishes scanning.
- Improved the web frontend to gracefully handle errors if the AI returns incomplete data.
- Designed an interactive "Attack Path" panel in VS Code to show exactly how the AI exploited a bug.

Work to be Done in Next Week:

- Do a full system run-through to find integration bugs and finalize the database update logic.
-

WEEK 13

Work Done in Current Week:

- Successfully completed Progress Evaluation 2 with the panel!
- Finished the webhook logic so the database automatically updates when a scan finishes.
- Spent most of the week doing full-lifecycle testing across the extension, web, and backend to iron out state bugs.

Work to be Done in Next Week:

- Add API rate limiting to prevent server overload, polish the UI, and start writing user docs.
-

WEEK 14

Work Done in Current Week:

- Added strict API rate limiting to the backend to protect our cloud compute resources.
- Cleaned up the IDE extension's user experience with smooth status bar notifications and better error dialogs.
- Optimized the AI's token usage and made the code parser more robust against syntax errors.

Work to be Done in Next Week:

- Write cron jobs to clean up old data, package the extension, and deploy the web app.
-

WEEK 15

Work Done in Current Week:

- Wrote automated cleanup tasks to delete sensitive code zip files from the cloud after a scan is done.
- Successfully deployed the web dashboard to production.
- Compiled the final `.vsix` binary for the VS Code extension so it's ready to install.

Work to be Done in Next Week:

- Run heavy stress tests on the production servers and finish writing our final project reports.
-

WEEK 16

Work Done in Current Week:

- Did final stress testing on the whole system to ensure the Docker sandboxes clean themselves up properly under load.
- Finished compiling all our structural documentation and wrapped up this progress diary.

Work to be Done in Next Week:

- Present our final project defense and live demonstrations for the multi-department evaluation panels.